

Conformant but Not Compatible: An Interoperability Study of Thirteen Identity Wallets

Maxime Mansiet
Verana Foundation*

September 2026

Abstract

Digital identity rests on a promise: a credential issued by one organisation can be held in any wallet and shown to any verifier, none of whom agreed anything in advance. We tested whether that holds.

We read the source of thirteen production identity wallets from governments, companies and open-source projects, and recorded what each can actually accept: identifier methods, key types, credential formats, query language, signature algorithms. Four of those five intersections are empty. The fifth holds one algorithm, and only because four wallets support nothing else. Restricting the corpus to the eleven wallets that share a protocol family leaves three of the five still empty, and conditioning further on the eight that resolve identifiers at all leaves those two empty too.

Wallets also anchor trust in incompatible ways. Six ship no trust entries at all; among those that combine a built-in list with a fetched one there are five mutually incompatible rules for which wins. One wallet in thirteen cryptographically ties fetched trust data to a key compiled into the binary.

We then integrated one shared trust layer into fourteen wallets and into services on both sides of the exchange, across two unrelated credential protocols. Six have an evidenced issue-then-present round trip on a device. The gate that separates a trusted party from an authorised one, which is the case the registry exists to decide, is demonstrated on three wallets, found failing open on two, evidenced only by end state on three, and unevidenced on the rest. Across five independent stacks, none of the failures we diagnosed was a case of two parties implementing the same wire format correctly and still disagreeing.

We use the measurements to derive seven requirements for a trust layer and evaluate five approaches against them, including our own, which fails several. Pairwise protocol interoperability is tested, and inside a profile it largely works. Pairwise trust decisions are tested by nobody, and the two specifications that govern the two sides of that decision each place the other side out of scope.

1 Introduction

The case for verifiable credentials rests on a specific claim. An organisation issues a credential. A person keeps it in a wallet of their choosing. Some other organisation later asks for it and accepts it. The issuer and the verifier never spoke, never signed anything, and may not know the other exists.

That last part is what we set out to test. We call it *collaboration*, and we mean something narrow:

*The author works on the Verana project at the Verana Foundation. Verana is used in Part II as the deployed example of a queryable trust layer, and is evaluated against the same criteria as every other approach, including the criteria it fails.

Two parties who have never coordinated can still transact. An issuer’s credential is accepted by a verifier it has no relationship with. A wallet holds credentials from two issuers that never heard of each other, and both are verified the same way.

Part I measures what is actually there, and it has two findings at two different layers. The first, in §4, is that the layers everybody does test still do not converge: no value of an identifier method, a key type, a credential format or a query language is accepted by all thirteen wallets. The second, in §5, is about the layer nobody tests, and it is the one Part II builds on. Wallets do not merely hold different trust lists; they disagree about what a trust anchor is and about which source wins when two of them disagree, and no specification tells them.

The first finding matters because it establishes that conformance and convergence are different things even where testing is dense. The second is what the rest of the paper is about, and it can be put in one sentence:

Where a client’s built-in assumptions meet what it learns at runtime, no specification says which wins, and every implementation answers differently. That undecided composition is prior coordination, surviving in the binary.

Contributions. (1) A source-level measurement of what thirteen shipping wallets accept across five dimensions, and the intersections (§4). (2) A taxonomy of how they anchor trust, with five incompatible composition rules (§5). (3) An integration study across fourteen wallets, two protocol families and all three roles, with six evidenced round trips and an honest account of what the other eight do and do not show (§6). (4) Seven requirements derived from those measurements, an argument that two of them become preconditions when the verdict’s consumer is a software agent, and an evaluation of five approaches against them (§7–8). (5) An account of where a deployed registry falls short, including a case where the system built to remove hardcoded trust lists reproduced one (§10).

Measured versus argued. Sections 4–6 report measurements; §4 is reproducible from public repositories once the per-cell file and line references are released with the artifacts. Sections 7 onward contain arguments, and where an argument follows structurally rather than empirically we say so.

2 Background

Four things must hold for two independently built systems to work together, and the existing test suites cover three of them (Figure 1).

The ecosystem does test itself, and it publishes the results. At the OpenID Foundation’s interoperability event of November 2025, covering Terbu et al. [76] and Lodderstedt et al. [54], wallet-and-verifier pairings passed 98% of tests over the browser’s Digital Credentials API (44 pairings) and 73% without it (11), and wallet-and-issuer pairings passed 82% (22) [61]. The European Commission, whose member states are obliged to offer a wallet under European Union [24], ran a first multi-day test event of over 570 peer-to-peer tests between 19 organisations from 16 countries, of which 420 passed, and 101 tests against its reference implementation, of which 44% passed [20]. An ETSI plugtest reported 81.6% across roughly 15,000 attestation-format tests.¹ All of it sits in the top three bands. The pairwise events test whether two implementations can exchange a message; none of them tests whether either party should have accepted it.

¹As reported by a participant; ETSI’s own report had not been published at the time of writing.

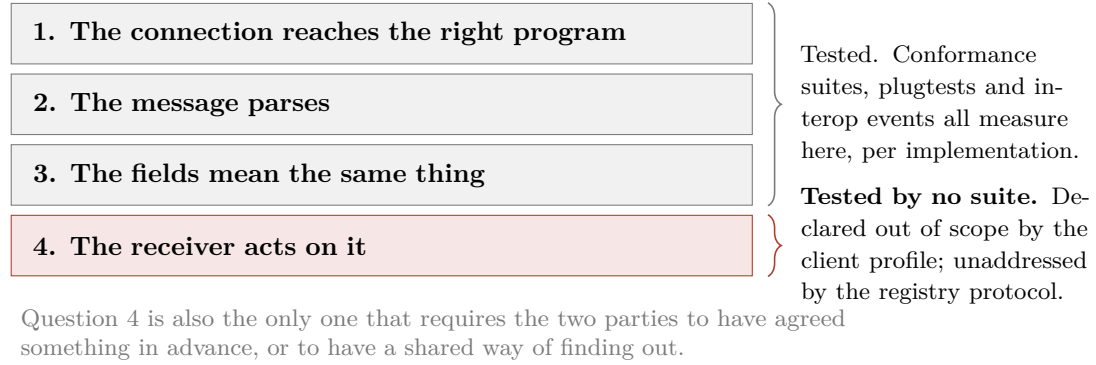


Figure 1: What has to line up, and what anybody checks.

That distinction is old. Castro [6] stated it for OSI systems in 1991: conformance is necessary for interoperability and not sufficient. The inference from conformance to interoperability is still made at the one layer nobody tests.

The gap at question 4 is documented by the standards bodies themselves. The high-assurance profile [85] states that trust management is out of scope and that methods for establishing trust or obtaining root certificates are out of scope; it specifies exactly how a wallet transports a certificate chain and says nothing about which roots to trust. The trust registry query protocol [44] standardises the registry side of the same question. Its scope statement covers the wire between a registry endpoint and a consumer, defines a conformant consumer as anything that sends well-formed queries, and says nothing about what the consumer does with the answer. One specification defines the client and declines the trust; the other defines the trust and stops at the wire.

The field has begun to say so itself. The Global Digital Collaboration conference of September 2026, whose organisers had formed a task force on trust registries earlier in the year, introduced it with a keynote whose published description reads: “standards for issuing and presenting credentials have matured, but no equivalent governance exists for determining who should be trusted to issue or rely on them” [32]. Another session’s description reduces the question to one line, is this issuer recognised to issue this and is that still true, and observes that “no credential format answers it” [32]. The same week, the mobile driving licence standards were made free to read [30]. The format layer keeps getting easier to reach. The trust layer does not.

3 Method

Reading the wallets. We selected thirteen implementations that reach real users, twelve of them applications and one a core library that ships inside one of the others, whose source is public, spanning mobile and browser platforms, five languages, and three kinds of origin. For each we recorded, with file and line references, what the code accepts: resolvable identifier methods [16, 70]; verification key types the key layer can map to a usable key; signature algorithms, separately for issuance and presentation where they differ; credential formats [28, 40, 71], distinguishing media types that are commonly conflated; query language; response modes, established by locating cryptographic classes rather than metadata strings; how the key signing issuer metadata is discovered; hard numeric limits; and registered link schemes. We recorded what the code does, not what the documentation or published metadata claims. In one case those differ in a way that matters (§4).

Anonymisation, and what it costs. Wallets appear as W1–W13, in an order unrelated to any alphabetical or chronological ordering, with stack and origin given (Table 1). Naming would

make the findings directly actionable and the measurement directly auditable, and the genre this paper belongs to normally names implementations. We do not, because the same corpus carries the configuration findings of §5, several of which are security-relevant and none of which we have put through a disclosure process. Publishing those against named products would be disclosure by paper. The cost is real and we state it rather than minimise it: a reviewer cannot check our mapping quickly, and the stack and origin columns are enough that a determined reader could reconstruct much of it anyway. This is a weaker position than either naming after disclosure or withholding the configuration findings, and it is the one we chose. We are running coordinated disclosure with the affected projects and will publish a named revision once that process closes.

Independence of the corpus. Thirteen entries are not thirteen independent codebases, and where they are not we say so. W7 is a core library rather than an application and W8 is a wallet built on it, so those two share an implementation of every dimension in §4 and should be read as one data point there; we keep them separate in §6, where they were integrated separately and cost different amounts. W2, W6 and W13 are built over the same TypeScript agent framework and inherit its resolver and key handling, which is why their identifier and key-type rows resemble each other. Shared upstreams make the intersections *more* likely to be non-empty, not less, so this does not inflate the headline result, but it does mean the effective number of independent implementations behind Figure 2 is nearer ten than thirteen.

The integration experiment. Separately we integrated one shared trust layer into fourteen wallets and into services on both sides. The integration corpus is the thirteen above plus one commercial fork of W3, which we label W14; it shares W3’s codebase but ships a different trust configuration, so it is a separate integration and not a separate measurement. The layer answers three questions over HTTP: is this identifier a registered service and who operates it; is it authorised to issue a given credential type; is it authorised to request one.

Success was defined in advance as six scenarios, stated as predicates so the study is falsifiable. S1: an accredited issuer offers; the wallet accepts and the credential is afterwards present. S2: a registered but unaccredited issuer offers; the wallet blocks the accept control. S3: an unregistered issuer offers; the wallet blocks the accept control. S4–S6 mirror these for presentation, with S4 presenting the credential obtained in S1.

S2 and S5 are the discriminating cases. They separate who a party is from what it is allowed to do. A system that conflates the two passes S1, S3, S4 and S6 while failing them.

Every scenario ran on a physical device against live services, and every gate was to be verified by performing the action rather than by reading whether a control reported itself enabled, because on several wallets that report is wrong. A later audit of our own capture scripts found that this rule was not enforced uniformly: two of the per-wallet drivers record a gate as held whenever the screen is unchanged after an attempt, including when no action control was located

Table 1: The corpus. Stacks and origins given; names not.

ID	Origin	Stack	ID	Origin	Stack
W1	Vendor	Dart/Flutter over Rust	W8	Vendor	RN over the W7 core
W2	Government	RN over TS agent framework	W9	Vendor	RN over TS SSI toolkit
W3	Government	Kotlin over a wallet core	W10	Government	Kotlin with Rust resolver
W4	Community	RN with native modules	W11	Vendor	Kotlin Multiplatform
W5	Government	Rust core, Flutter UI	W12	Community	Browser PWA
W6	Vendor	RN over TS agent framework	W13	Vendor	RN over TS agent framework
W7	Vendor	Rust core library			

	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	all 13?
Identifier methods														
web	■	■	-	■	-	■	■	■	■	-	-	-	■	∅
key	■	-	-	-	-	■	■	■	■	-	-	■	■	∅
webvh	-	■	-	-	-	■	■	■	■	■	-	-	■	∅
jwk	-	-	-	-	-	■	■	■	■	-	-	-	■	∅
none at all	-	-	■	-	■	-	-	-	-	-	■	-	-	
Verification key types														
Ed25519 2018	■	■	-	■	-	■	-	-	■	-	-	-	■	∅
Ed25519 2020	-	■	-	-	-	■	-	-	■	■	-	-	■	∅
JsonWebKey2020	-	■	-	-	-	■	■	■	■	-	-	-	■	∅
Multikey	-	■	-	-	-	■	-	-	-	-	-	-	■	∅
Secp256k1	■	-	-	-	-	-	-	-	■	-	-	-	-	∅
Credential formats														
SD-JWT VC	■	-	■	■	■	■	■	■	■	■	■	-	-	∅
mdoc	-	-	■	■	■	■	■	■	■	-	■	■	-	∅
W3C VCDM	■	■	-	■	-	■	■	■	■	-	■	-	■	∅
AnonCreds	-	■	-	-	-	■	-	-	-	-	-	-	■	∅
Presentation query														
DCQL	-	-	■	-	■	■	■	■	■	-	■	■	-	∅
Presentation Exch.	■	-	■	■	-	■	■	■	■	■	■	-	-	∅
Signature algorithms														
ES256	■	■	■	■	■	■	■	■	■	■	■	■	■	yes
EdDSA	■	■	-	■	-	■	■	■	■	■	-	-	■	∅
ES256K	■	-	-	■	-	-	-	-	■	-	-	-	-	∅

Figure 2: What each wallet accepts. A filled square is support found in code; a dash is its absence. Only one row is complete, and only because four wallets support nothing else.

and no tap was issued, and one never taps at all in block mode. We therefore distinguish throughout between cells where the action was attempted and cells where only the end state is on record, and we do not treat the second kind as a demonstration that a gate holds.

4 What the wallets actually support

Figure 2 is the measurement. Figure 3 is its summary.

Four intersections are empty. The fifth is a single element, and it is not convergence: four wallets support nothing else. The mechanisms differ. One pins the algorithm in the default header of every token it builds. One bakes it into its format declarations. Others pass it as a constant when importing the issuer’s public key, so a credential signed with any other conformant algorithm fails with a generic verification error rather than a negotiation failure.

We did not run a conformance suite against these wallets and make no claim about whether they pass one. What the matrix shows is narrower and is what the rest of the paper rests on: each of these values is a legal choice under the specification the wallet implements, so an implementation can be entirely correct and still share no common value with another that is also correct.

Restricting to one protocol family. Two of the thirteen implement no OpenID4VC at all. They exchange credentials over peer-to-peer messaging in a different format, and including them could be said to rig the result, since nobody expects a wallet from one protocol family

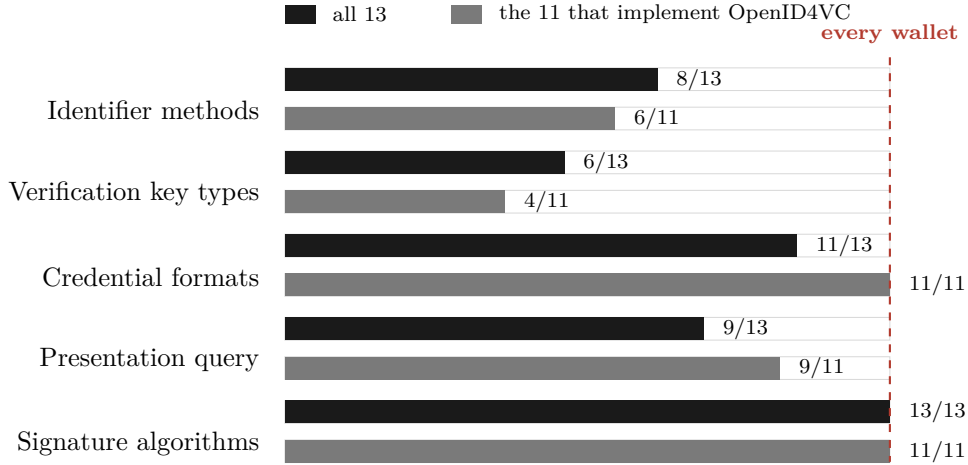


Figure 3: For each dimension, how many wallets support its *most widely supported* value, over the whole corpus and over the eleven that share a protocol family. A bar reaching the line means the intersection is non-empty. Restricting to one protocol family closes credential format, on SD-JWT VC, and leaves the other three open. Denominators count W7 and W8 separately although they share a codebase; merging them changes no intersection, since their rows are identical.

to interoperate with one from the other. So we recomputed the intersections over the eleven OpenID4VC wallets alone, shown as the second bar in Figure 3.

Three of the five are still empty. What changes is credential format: all eleven parse SD-JWT VC [28, 77], which is the one place the ecosystem has genuinely converged. The protocol families explain one of the four empty intersections. They do not explain the other three, and the two dimensions that decide whether a party can be named and its key used at all are the two that degrade furthest, to 6 and 4 wallets out of eleven.

Those two figures could be read as counting wallets to which the dimension does not apply, since three of the eleven resolve no identifier method at all. They do not survive that objection either: conditioning further on the eight OpenID4VC wallets that resolve identifiers, the most widely supported method reaches 6 of 8 and the most widely supported key type 4 of 8. Both intersections stay empty on every subset we can construct.

Two wallets with no overlap at all. One wallet built on a Rust core emits and consumes exactly one key type. One built on a Dart credential library maps exactly two types to usable keys, and the first’s type is not among them. A well-formed identifier document produced by one is unusable by the other, and neither violates any specification.

A wallet rewriting documents to survive. The key type that the identifier specifications increasingly prefer is supported by fewer implementations than the older types it replaced. One wallet ships a function that rewrites identifier documents in flight, retyping the preferred type into an older one, because its own key layer has no case for it. The comment notes the substitution is lossless for the two encodings it handles and that unknown encodings fail downstream. That is a translation layer written by hand for a case no suite tests, because the suites ask whether an implementation can resolve an identifier, not whether it can use what it resolved.

Rejected for being interoperable. One wallet core treats the three ways of conveying a signing key as mutually exclusive and returns an error describing the input as a mixed specification of public key information when more than one is present. An issuer serving both

the certificate-oriented and identifier-oriented wallets has an obvious move, which is to supply both. That issuer is rejected specifically for having made the interoperable choice. Nothing forbids supplying both; nothing tests for it.

Advertised is not implemented. One wallet publishes metadata declaring encrypted authorization responses and naming a key agreement algorithm. There is no encryption implementation anywhere in its source. Five of thirteen have a genuine implementation, established by locating the cryptographic classes; among those five the key agreement and content encryption intersections are one algorithm each, and two of the five still do not negotiate because one requires a key wrapping variant the other rejects.

Failure before the protocol. One link scheme is registered by seven of thirteen wallets. On one mobile platform this produces a disambiguation prompt; on the other the behaviour is undefined and the last application installed wins. Two conformant wallets on one device means the operating system may route the invocation to the wrong process before a single line of conformance-tested code runs. We also observed one wallet registering the scheme belonging to another country’s national wallet.

5 How wallets decide who to trust

We expected thirteen hardcoded lists. Six of the thirteen ship no trust entries at all; the others ship between one and thirteen. Seven have both a built-in list and a runtime mechanism, and there are five mutually incompatible rules for how the two combine (Table 2).

Table 2: Five rules for combining a built-in trust list with a fetched one.

Rule	Behaviour
Built-in wins	Built-in anchor evaluated first, short-circuits to trusted. Fetched lists can add trust, never remove it.
Fetched wins, gated	Built-in config is a bootstrap; a fetched config replaces the anchor set wholesale, but only if signed by a compiled-in key and only if its version exceeds the current one.
Built-in gates fetched	Built-in identifiers authenticate the registry; a statement not signed by a pinned identifier is discarded.
Union, built-in immutable	Built-in and user anchors concatenated, built-ins first, no path to remove a built-in.
Static first, runtime fallback	Mechanisms tried in declared order; the registry query runs only when the static check finds nothing.

One wallet of thirteen cryptographically binds its fetched trust data to a key compiled into the binary. The rest either do not fetch, or fetch over a transport-secured channel from a compiled-in address.

This is the more important of Part I’s two findings. **The wallets do not merely disagree about who to trust. They disagree about what a trust anchor is, and about what it means for one source of trust to override another.** No test suite has a concept of that, because no specification defines it. Where a specification is silent, implementations do not converge on a safe reading; they converge on whatever their own history makes convenient, which is the argument the IETF eventually made against tolerating unspecified behaviour [79].

What is currently normal. Reported without attribution: a widely embedded core ships an empty anchor array with a comment stating that empty means trust on first use, accepts the trust list’s own signer when nothing is pinned, and permits five levels of chained pointers

from that unauthenticated root. One wallet’s chain validation supplies, as the set of trusted certificates, the root of the chain it is validating, so the check is self-referential; its fallback handler renders the requesting party’s self-declared name and logo with an empty trusted-entity set. One wallet contains a function validating a chain against an empty trusted list, with no caller anywhere in its source, and a configuration flag that appears to enable trust anchor validation but is read once and never referenced. One ships two certificates that expired before the build we examined, alongside an anchor mode accepting a self-signed leaf and a facility for adding anchors by scanning a code. One consortium trust list is served unsigned from a static site host. One wallet lets the user repoint its entire issuer registry from a settings screen, which is simultaneously the strongest user agency and the weakest anchor in the corpus.

6 The experiment

We integrated one shared trust layer into fourteen wallets and into services on both sides. The layer is a public registry with an HTTP query interface, backed by a public ledger; any party can ask whether an identifier is a registered service and whether it is authorised to issue or request a credential type, and any third party can query the same interface or re-derive the answer.

The corpus spans two credential protocol families sharing no code, and all three roles. Table 3 is what we can actually evidence, and it is less than we expected when we began writing this section.

Table 3: Per-wallet scenario outcomes. Capitals mark evidence of the action itself, lower case marks the end state only. **B** the action was attempted and refused; **b** a refusal is on record without the action being attempted; **P** the flow completed and the result was checked; **p** the flow is recorded as complete without that check; **W** the gate failed open; – not evidenced; **X** could not be run. W14 is a product fork of W3 and is in the integration corpus only. S2 and S5 are the discriminating cells.

	Rail	S1	S2	S3	S4	S5	S6	Round trip
W4	sd-jwt (PE)	P	b	b	P	b	b	evidenced
W3	sd-jwt (x5c)	W	–	–	P	W	P	regressed
W14	sd-jwt (x5c)	P	W	P	P	W	P	evidenced
W6	sd-jwt	p	–	–	p	–	–	not evidenced
W8	sd-jwt	P*	B*	P	P	B	P	evidenced
W10	sd-jwt (x5c)	P	B	P	P	B	P	evidenced
W1	sd-jwt (PE)	P	B	P	p	B	P	not evidenced
W9	sd-jwt	p	b	b	p	b	b	not evidenced
W13	anoncreds	P	b	b	P	b	b	evidenced
W2	anoncreds	–	–	–	–	–	–	not evidenced
W12	browser	–	–	–	–	–	–	not evidenced
W5	sd-jwt	X	X	X	X	X	X	blocked
W11	sd-jwt	–	–	–	–	–	–	not evidenced

* W8’s issuance gate is advisory: the credential is fetched and stored before a verdict exists, so S2 blocks the accept control while the credential still lands in storage.

Six of the fourteen have an evidenced round trip, meaning a credential obtained in S1 was afterwards presented and accepted in S4 on a physical device, with both ends on record. Five are marked *evidenced* in Table 3; the sixth is W3, marked *regressed* because the round trip itself completed while its issuance gate failed open. Three more were signed off internally against a definition of done that requires the round trip, but no line in our record states that the presentation completed, so we do not count them. One wallet is blocked outright: its core requires a certificate chain that does not contain its own trust anchor, and our demo services

sign with a single self-signed leaf, so it needs a two-level public-key infrastructure and a hosted provider we did not stand up. The rest were integrated in code and never driven end to end on a device.

An earlier draft of this paper said twelve of fourteen. That number counted integrations that had been declared complete rather than round trips we could point at, and it did not survive going back through the evidence.

The discriminating cells are the weakest part of the study. S2 and S5 are where the study was supposed to be falsifiable, and they are the cells we can least support. Three wallets refused the action on S2 when it was attempted, and the same three on S5. Three more have a refusal on record without the action being attempted, which evidences the end state and not the gate. Four have no S2 or S5 evidence at all, one could not be run, and one is a core library we did not drive through a user interface. Two were verified *wrong*: on one run the accreditation gate failed open on both cells and the wallet accepted a credential from a service its own trust card described as unauthorised. That defect and its cause are the subject of §6.1. It was fixed in source and the fix was covered by unit tests, but the fixed build was never re-run on a device, so this paper reports the failure and not the remedy.

The honest summary is that the positive path is demonstrated across two protocol families and three roles, and the gate that separates identity from authorisation is demonstrated on three wallets, contradicted on two, evidenced by end state alone on three, unknown on four, blocked on one, and not driven on the core library.

What one integration takes. There is no single place to put a trust check. We found six architecturally distinct insertion points: a callback on an agent framework’s configuration; a call site between request verification and response construction; a trait implementation in a Rust core; a policy plugin in a verification pipeline; a database flag and table in a server backend; and a browser script intercepting network calls made by an unmodified application. That last one is the only integration requiring no fork and no cooperation, and we return to it in §11.

Table 4 is what it cost. We computed it from each fork’s history, diffing against the nearest true upstream and excluding lockfiles, build output, binaries and generated snapshots. One bridge package of 196 lines is folded into W8. The three-way split is a path heuristic: a file counts as interface if its path names a view, screen, component, widget, style or theme, as tests if its path names a test, and as logic otherwise. We did not classify files by reading them, and the figures below should be read with that in mind.

These are lines *we* wrote, so the table measures what it costs to add one shared trust source to a client, not how much trust code the wallets already contained. Read that way, two things follow, and the second is not the one we expected.

First, **presenting the verdict costs about as much as computing it.** The median integration wrote 994 lines of trust logic and 672 lines of interface, and in five of the thirteen the interface was the larger of the two. The decision itself is a few hundred lines; making it legible to a person is a comparable engineering problem, and we solved it separately thirteen times because the thirteen user interfaces have nothing in common.

Second, **a shared trust source does not remove the per-client cost; it is that cost.** About 15,700 lines of logic went into thirteen clients to ask one service the same three questions. None of it is shareable: a Dart module does not compile into a Kotlin application, and a Rust core does not present a screen. Centralising the *answer* leaves the work of asking, interpreting and displaying it distributed across every client that wants it. Any argument that a registry reduces integration effort has to be made against this number, and on this evidence it does not hold; what a registry changes is who can be asked about and when, not how much client code exists.

Table 4: Cost of one integration, in lines added. *Logic* is trust resolution, parsing and gating; *UI* is presenting the verdict to the user; *Tests* is the trust path’s own tests. The split is assigned by file path rather than by reading each file, so *Logic* is the default bucket and absorbs configuration and build changes. Every wallet answers the same three questions.

	Logic	UI	Tests	Files	Commits	Inserted at
W1	2 063	1 595	3 260	53	34	app module
W2	528	329	0	23	6	app component
W3	994	672	0	28	9	core controller
W4	1 568	1 400	930	49	23	app module
W5	589	150	721	29	4	data layer
W6	2 347	620	0	35	21	framework SDK
W7	2 577	0	702	37	7	core library
W8	587	1 497	667	26	15	app, over W7
W9	1 043	1 295	500	32	9	app service
W10	1 999	1 733	1 959	79	13	platform module
W11	255	275	0	11	1	shared service
W12	358	431	0	11	1	service hook
W13	775	823	574	36	11	app service
total	15 683	10 820	9 313	449	154	
median	994	672	574			

The tests column is ours as well and says nothing about these products’ own coverage. What it does show is how unevenly *we* treated the same security decision across thirteen ports: five got no trust-path tests at all, three got more test code than logic. The variance tracks how the fork was run rather than anything about the wallet: the two ports with a single commit each wrote no tests, and the port with thirty-four commits wrote the most.

A separate observation, recorded because it recurs and appears in the literature only once [59]: projects in this corpus describe themselves as straightforward to integrate with, and the integration rarely was. The gap was rarely in the protocol. It was build toolchains, undocumented configuration, stale instructions, and behaviour differing between build flavours.

What broke. We diagnosed failures across five independently developed stacks. None was a case of two parties implementing the same wire format correctly and still disagreeing; each traces to one side departing from, or over-constraining, what the specification says.

Identifier semantics

Two conformant implementations of one identifier method derived the identifier differently. The cause was a field name spelled in the plural by one library and the singular by the specification, silently dropping a value before a hash. The affected library rejected its own project’s reference fixture when run through the other pipeline.

Algorithms advertised but not implemented

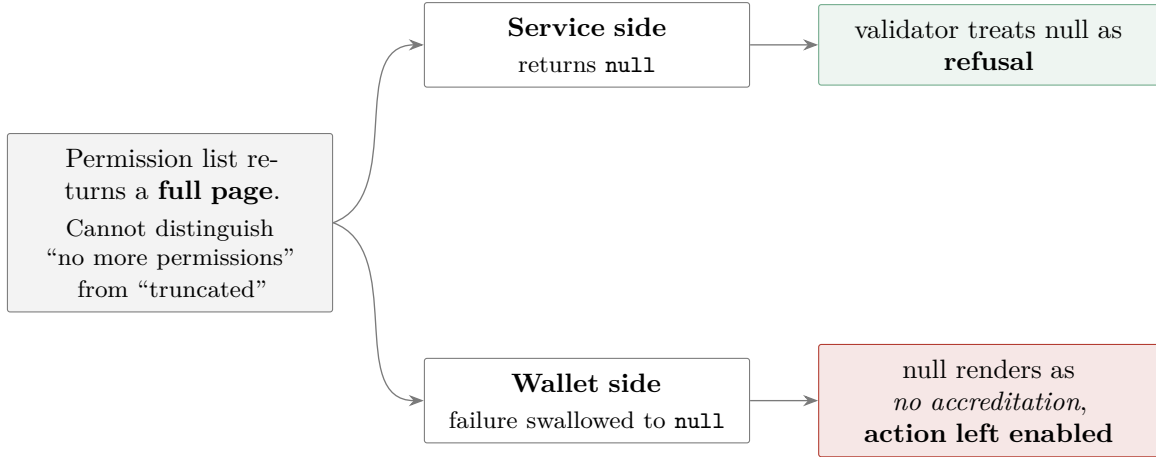
In one case the advertised algorithm sets matched exactly on both sides and the exchange still failed, because the receiver accepted only one key type for that algorithm and the sender used another.

Where a key is discovered

Some require a certificate chain in the header, some an identifier. Serving both is possible, since the fields are independent, and we did that. One implementation rejects tokens carrying both.

Resource ceilings

One wallet refused a cryptographically sound identifier because its log had grown past a



Same author, same registry, same defect class. The service integration is *additive*: it can turn a rejection into an acceptance and never the reverse, so an outage degrades to prior behaviour. The wallet check *replaces* a decision, and there is no prior behaviour to fall back to.

Figure 4: One condition, two integrations, opposite outcomes. The difference is structural, not a matter of care.

compiled-in maximum. The refusal happened before any protocol ran, and the log grows by roughly ten entries a day.

Observability

Several release builds emit nothing on the failing path, and on more than one wallet the accessibility tree reports a control as enabled when it is not.

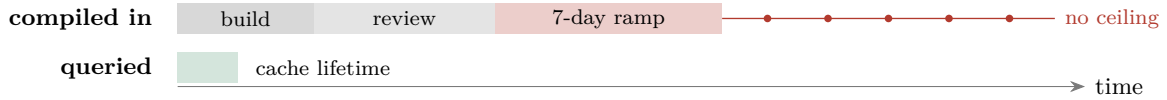
6.1 The same defect, two outcomes

The most instructive result here is a comparison we did not set out to make (Figure 4).

The registry’s list query returns at most a fixed number of records with no cursor, so a caller receiving a full page cannot tell whether the party has no further permissions or the page was truncated. We wrote handling for that condition twice, on opposite sides of the wire, with the outcomes shown. The wallet displayed a trusted verdict with acceptance enabled for a service whose own description states its offers must be refused.

The explanation generalises. Both service-side integrations are *additive* and default-off: consulted only after the native check has already refused, so they can only convert a rejection into an acceptance, and an unconfigured deployment is unchanged. A wallet-side check has neither property. It replaces a decision, and there is no prior behaviour to degrade to because the wallet had no opinion about this issuer before. The first shape degrades to prior behaviour when the registry is unavailable; the second replaces the decision outright, and did fail open.

The additive shape does not prevent a wrong acceptance: converting a native refusal into an acceptance is exactly what S2 exists to test. What it bounds is the failure mode when the registry cannot be reached. This is an old principle [66] arriving at a new layer. We saw both shapes chosen by the same author on two integrations, so we report it as a design option rather than as a property of either team.



Reported figures for the tail: roughly half of users had not updated a week after a fix shipped [58]; applications ship an upstream security patch a mean of 528.7 days after it becomes available, against a 54.6-day mean for the library itself [2]; 85.6% of vulnerable library versions were updatable with no application code change [15], so the delay is not technical.

Figure 5: Changing a trust decision. A floor set by review and staged rollout, a tail set by adoption, and no upper bound at all.

Part II: Requirements derived from the measurements

7 Requirements

Four of these requirements are derived from measurements reported above. Three are arguments, and each says so in its own derivation line.

R1. Independence from the stack.

It must not depend on any identifier method, key type, credential format or query language. *From: four empty intersections. There is no common substrate.*

R2. Defined composition.

It must specify how its answers combine with the client’s existing trust mechanism. *From: five incompatible composition rules, none written down.*

R3. Bindable answers.

A client must be able to bind answers to something it already trusts. *From: one wallet in thirteen does this.*

R4. Answers at decision time.

It must answer when the decision is made, not when the client was built. *From: the argument below. Every wallet in our corpus that carries a built-in list also has a runtime path, so this requirement is about which source governs the decision, not about lists that cannot change at all.*

R5. A third answer, with a defined meaning.

It must be able to say it does not know, and the specification must say what the client does then. *From: every wallet in the corpus mishandled this state.*

R6. Answerable without prior coordination.

Two parties who never interacted must be able to get an answer about each other. *From: the definition in §1.*

R7. Auditable by a non-operator.

A third party must be able to check the answers, or the client has swapped one opaque authority for another. *From: the trust displacement argument, §10.*

7.1 On R4

R4 rests on an argument, not a measurement of ours, and we mark it as such (Figure 5).

Table 5: Where five agent specifications anchor trust and what they do with the unknown case. Every cell is taken from the cited text or its reference code. The last column is whether composition with the client’s existing trust is defined (R2). None of these documents sets out to be a trust-composition specification, so a uniform *no* is not an indictment of any of them; it records that the layer is unclaimed here exactly as it was in §5.

	Anchor	Unknown or unreachable	R2
AP2 [35]	a credential issuer, or a verifier-local “trust list of Agent Providers”	an unknown constraint MUST fail; the error may trigger a non-agent fallback	no
Verifiable Intent [82]	the signer’s published key set	default mode PERMISSIVE: unknown constraint types skipped	no
Visa TAP [83]	a fetched key directory; the reference merchant hard-codes two	failed fetch and unknown key refused alike	no
Web Bot Auth [57]	a fetched key directory	third outcome <i>unverified</i> ; a failed fetch is not evidence; handling is local policy	no
A2A [1]	optional card signatures; clients MAY keep a trusted key store	not defined	no

When the set of acceptable issuers is compiled into an application, changing it requires publishing a new build. We are not aware of any published measurement of this bound for trust material specifically. The certificate pinning literature states the mechanism, that rotating a pin requires a release [60, 65], without measuring the latency. The browser root store is the same object one layer up, a trust list shipped with software, and the stores shipped by different vendors have been shown to diverge from one another [46]; two decades of proposals to improve on that arrangement are surveyed by Clark and van Oorschot [11]. The web PKI reached the same conclusion by another route: after measurements showed revocation checking unreliable in practice [10, 53], the response was transparency logs and shorter lifetimes rather than better push-based revocation, and pushing the full revocation set to every client needed a research contribution of its own [50].

7.2 On R2 and R5: when the verdict’s consumer is an agent

This subsection is an argument. It rests on reading specifications and reference code, not on measuring deployed agents.

The consumer of every trust verdict in Part I was a person. The wallet drew a card, the person read it, and the two things nobody had specified, how the registry’s answer combines with what the wallet already trusted and what “could not determine” means, were absorbed by someone who could hesitate. Payment networks and platform vendors now propose that software agents transact on a person’s behalf, and the published description of one conference session on the subject states that the technologies involved “leverage the wallet as the Trusted Surface” rather than “introducing a new trust model” [32]. An agent can be built to stop and ask, but its hesitation is a designed feature at a chosen point, and it has to be triggered by something the code can name. A wallet can render an unexplained verdict and let a person supply the caution; an agent needs the ambiguity represented in the interface before it can route it anywhere. A composition rule that is not written down executes as whatever the code happens to do, and a third answer with no defined meaning collapses into one of the other two before anything can pause on it. For a person, R2 and R5 are requirements. For an agent they are preconditions.

The agent stack that exists today repeats Part I’s pattern (Table 5). Google’s payments protocol requires that “any unknown Constraints MUST be treated as failing evaluation” [35]. The intent specification Mastercard and Google published the following March defaults to a

mode that “silently skips unknown constraint types” [82]. Visa’s reference merchant hard-codes its trusted directories in source, under a comment that in production they would be loaded from configuration, and its reference proxy answers a failed registry fetch with the same refusal it gives an unknown key [83]. The one text that keeps the third state distinct is a bot-authentication draft: it separates *verified*, *invalid* and *unverified*, states that “a failed fetch says nothing about the signer” and that a directory outage “MUST NOT evict a cached entry”, and then leaves what to do with an unverified request to local policy [57]. Two opposite defaults for the unknown case, one compiled-in list, one three-valued outcome with no required behaviour: that is the composition table of §5 one layer up, written in the past eighteen months by the best-resourced parties in the industry.

Two consequences follow. First, the cost finding of §6 changes shape, not size. The median wallet spent 672 lines showing a verdict to a person; an agent needs the same verdict as a typed result with defined semantics, which is an interface to specify once rather than a screen to draw thirteen times. Second, R5 must demand a specified behaviour rather than a fixed one. Chen [9] measure that an agent which verifies every counterparty on every interaction fails through indecision, so “unknown means deny” is not free. The requirement is that the specification say what the client does; the wallet-side failures of §6.1 are what happens when it does not.

We know of no published measurement of what a deployed agent does when the trust source it depends on answers “unknown” or does not answer at all. The closest work is discussed under related work, and none of it composes a registry’s answer about a counterparty with a client’s prior anchor, which is what R2 asks.

8 Approaches evaluated against the requirements

Table 6: Five approaches against the requirements. A dash marks a requirement the approach does not address rather than fails.

	R1	R2	R3	R4	R5	R6	R7
Compiled-in list	yes	–	yes	no	–	no	no
Curated trust lists [19, 21]	yes	no	yes	partly	no	partly	partly
Federation	no	no	yes	yes	no	partly	yes
Name-anchored	yes	no	partly	yes	no	partly	partly
Queryable registry	yes	no	partly	yes	partly	yes	partly

Two things in the table are worth drawing out.

First, **no approach satisfies R2 or R5**. Composition and the third answer are unaddressed across the entire design space, which is consistent with Part I, where we found five incompatible composition rules and universal mishandling of the unknown state. No single design is at fault; neither requirement has been specified anywhere we could find.

Second, federation [62] is the strongest competitor. It resolves chains at request time with short-lived statements, so it is not vulnerable to R4 at all. It differs from a registry in what can be asked: it answers whether an entity belongs to a federation and what metadata it may assert, rather than an arbitrary question about authorisation for an action on a resource. It also requires every participant to operate a signing endpoint, a real operational cost a registry does not impose.

9 A queryable registry in practice

The layer we integrated is a public registry with three query endpoints, backed by a public ledger and served by an indexing service. Ecosystems register on the ledger; a schema owner grants

issuing or verifying authority directly or through an intermediate grantor; any party can query whether an identifier holds a given authority for a given credential type.

Entry is not gated. Any account can register an organisation, and any registered organisation can create an ecosystem and define schemas. Participation carries a deposit, but the protocol does not set its size: the deposit is derived from a validation fee chosen by the ecosystem being joined, and that fee may be zero. Creating an organisation, ecosystem, schema or governance document carries no deposit at all.

The read interface requires no authentication and imposes no rate limit. All registry state is ledger state, so an independent party can re-derive it, and the evaluation library is published under an open licence at roughly two thousand lines.

That last point matters for R7 and we want to be precise. The answers are auditable in that the inputs are public and the evaluation code is readable. They are not independently attested: one service computes the verdict, nothing requires two indexers to agree, and the verdict is not published back to the ledger.

10 Where it falls short

10.1 The list came back

The evaluation service carries an operator-configured list of ecosystem identifiers permitted to define the essential schemas the trust rule depends on. Its own documentation explains what happens when that list is left empty, which is the default: any ecosystem qualifies, and because the reference schema digests are public and fixed, an unlisted ecosystem can publish a byte-identical schema, authorise itself as its issuer, issue itself the credentials the evaluation looks for, and be reported as trusted. The barrier, in the documentation’s own words, is the deposit rather than the cryptography.

A system built to remove trust lists compiled into applications reproduced one, in an environment variable, one layer up.

We report this because we think it is the most useful thing in this section, and we do not think it is a defect of implementation. Trust has to be rooted somewhere; it cannot be eliminated, only relocated. The question a design can answer is not whether there is a list, but whether the list can be seen, compared between operators, and changed without shipping software. That is consistent with the displacement literature [4, 13, 48], which argues that systems presented as removing the need for trust relocate it somewhere less accountable. Our finding is that this happened to us, in a system designed with the opposite intent, so the honest claim is narrower than the one usually made.

10.2 Other shortfalls

- **R2 unmet.** The specification says how to display an unknown result and not whether the client may proceed, and carries an unresolved editorial marker on exactly that question. This is the specification-level cause of the wallet failure in §6.1.
- **R5 partly unmet.** The service-side specification names resolver unavailability as a first-class verdict and requires failing closed. The client-side guidance does not.
- **No federation primitive.** Ecosystem-to-ecosystem recognition is out of scope in the version we used, so the cross-border claim in §11 is an argument about what the architecture permits, not a report of something we ran. The gap is not ours alone. The collaborative notes of a September 2026 conference session on trust registries record that recognition between independently governed registries “is a goal, not a solved problem”, and that the first governance framework of the conference’s own task force is “an early, deliberately narrow draft focused on government issuers of identity documents” [31].

- **One indexer.** See §9.
- **Pagination.** The ledger query interface has no cursor and caps responses at a fixed size, the condition behind §6.1. We report one figure we cannot fully reconcile: the record set we measured held 845 entries against a cap of 1024, so that particular query was not returning a full page. Either a different and smaller set was truncated on the failing run, or the fault lies in the handling of the capped response rather than in truncation itself. We did not re-run the failing build to settle it.
- **Specification ahead of implementation.** Several described capabilities are absent from deployed code: a partial trust state no code path emits, permission chain details returned as empty objects, and query protocol conformance that is on the roadmap. Where this paper describes behaviour, it describes what runs.

11 Interoperability without prior coordination

A compiled-in trust list is prior coordination in durable form. Someone decided, before the software shipped, which parties would be acceptable. Every transaction it permits was arranged in advance through an intermediary, with neither party present. That is a legitimate way to build a system. It is not the property verifiable credentials are usually claimed to have.

Part I says the ecosystem has been assuming the property without providing it. The empty intersections do not say that two implementations picked at random cannot transact; inside a profile many pairs can, and the events in §2 show them doing it. What they say is that an issuer cannot make one choice that reaches all of them, so reaching a wallet still means knowing in advance which wallet it is. Five incompatible composition rules say that even where a runtime mechanism exists, no two clients combine it with their prior assumptions the same way. And the layer where that would be settled is out of scope for both the profile governing clients and the protocol governing registries.

One existence proof is worth taking seriously. A research and education federation interconnects 84 national federations and more than ten thousand entities through a shared layer [37]. Cross-jurisdictional trust establishment scales past eighty authorities.

The same deployment is also a caution, in three ways. It is not peer-to-peer recognition but a central signed aggregate with one root, identified as a scalability and reliability concern [36]. Membership is not a working service: a community requirements effort exists precisely because federation membership did not deliver usable cross-domain access [3], and the response was an adapter layer, which is the pairwise integration cost reappearing one level up. Farrell and Saloner [27] give the reason: partial converters preserve fragmentation rather than dissolving it. And that federation is single-protocol. Everyone speaks the same protocol, and trust still took roughly two decades of separate policy work after the protocol problem was solved. A two-protocol ecosystem should not expect trust to fall out of protocol convergence.

A second precedent points the same way. The registry through which states publish their passport signing keys lists 111 participants out of 193 member states after nearly two decades of operation [39]. Pre-arranged registry trust scales, and it scales slowly.

It is fair to ask what the property is worth to the person holding the credential. A passport and a boarding pass already cross borders. The answer is not speed. The European Union's entry and exit system is fully digital and biometric and has been fully operational since April 2026 [23]; within days the airlines' association reported three-hour queues at border control and asked for it to be suspended until the end of summer [72]. The bottleneck was process, not the document. What a credential changes is who has to have agreed in advance. A paper document is accepted where its issuer is already known and is a photocopy everywhere else. A credential whose issuer any verifier can check at the moment of presentation is accepted where its issuer has never been heard of. That is the capability the paper version lacks, and it is exactly the one

a compiled-in trust list gives away.

Which returns us to the one integration in our corpus that required no cooperation. It was performed against unmodified official images of another project’s components, by adding a browser-side script that observes the application’s own network calls and consults the registry before the user acts. Neither side was forked and neither vendor’s permission was needed.

It is not a deployment anyone should copy: a script that intercepts another application’s network calls is, structurally, a man in the browser, and it required the operator of that deployment to install it, which is a coordination step of its own. What it demonstrates is narrower. The trust question was answerable about a party that had never heard of the asker, by a third party, at the moment it was asked, without either vendor changing anything. Every other integration in this paper needed a fork. That one needed a network call, and that difference is the property Part II is about.

12 Limitations

An intersection is a strict measure. That no value is accepted by all thirteen does not mean no two of them can transact: many pairs share enough to complete an exchange, and the pairwise events in §2 show that they do. What an empty intersection rules out is a single choice an issuer can make that reaches every wallet, which is the property an issuer needs when it does not know who will present its credential. Figure 3 is the sensitivity check on the corpus; we did not run one on the dimensions, which we chose because they are the ones a message must satisfy to be read at all.

The corpus is a convenience sample and should be described as one. There is no sampling frame: these are the open-source wallets a single project could obtain, build and drive on a device in six weeks, which selects for open source, for active repositories, and for buildable toolchains. The wallets with the largest real user populations are proprietary and absent. Nothing here generalises to them, and the divergence we report may be larger or smaller in that population for reasons we cannot see.

Source-level analysis establishes what code paths exist, not what a deployed configuration does, and in a few cases we report a default rather than an observed production setting. A value present in a matcher is not proof that an exchange using it would succeed; the one case where we checked advertisement against implementation, we found them to differ, which is a reason to treat the whole matrix as an upper bound on capability rather than a measure of behaviour. The stronger version of this study runs each pair and reports what happened, and we did not do that.

The integration study is the weaker half of this paper and we would rather say so than have it read as stronger than it is. Six evidenced round trips out of fourteen integrations is a modest result; the discriminating cells are evidenced on three wallets and contradicted on two; our own capture harness was found, after the fact, to be able to record a gate as holding without having attempted the action. We have marked those cells rather than dropped them, but a reader should treat every lower-case cell in Table 3 as an observation about a screen and not about a gate. The correct version of this experiment re-runs all six scenarios on current builds with a harness that fails loudly when it cannot find the control it is meant to press.

We tested one trust layer, so nothing here supports a claim about queryable registries in general. Everything ran against a test network. The integrated wallets are our own forks, so the reported integration costs are what it cost us, with our familiarity, and not a neutral estimate of what an unfamiliar developer would spend. Anonymisation limits direct verification. The comparison of agent specifications in §7.2 reads specifications and reference code, not deployed agents, and there is no measurement of agent behaviour on an unreachable trust source to check it against.

13 Related work

The method belongs to an established genre: take many independently built implementations of one specification, probe them uniformly, report systematic divergence [18, 25, 33, 68]. Differential testing [5] and protocol state inference [14] use disagreement between implementations as an oracle, which is the move we make when we take an intersection. We follow Egele et al. [18] in stating success criteria as numbered predicates before reporting results.

On interoperability, Yildiz et al. [87] give the decomposition our four questions follow and Castro [6] the conformance result we rely on. Kubicek et al. [49] study 77 European cases and find the organisational layer, not the technical one, determines whether interoperation occurs. Tolk and Muguira [80] and Wang et al. [84] provide the formal model in which our fourth question is a composability rather than interoperability property. Chen et al. [8] classify the available responses as integrated, unified or federated, a distinction that separates the approaches in §8 more cleanly than any of them describe themselves. Gallaher et al. [29] put a figure on what inadequate interoperability costs one industry, and Litwin et al. [52] state the pairwise problem in its original form.

Two results predict what we found. Suzuki and Abe [74] argue that interoperability is not format compatibility but the condition that a verifier can obtain the materials it needs and reach a decision under stated assumptions; our measurement is the empirical form of that claim, and the two dimensions that degrade furthest in Figure 3 are the ones their model predicts. Strnadl [73] formalise interoperability extent as the commonality of trust profiles, which is an intersection over trust assumptions rather than over accepted values. Kramer and Moesel [47] show profile proliferation destroying interoperability inside a single standard, which is what the interoperability profiles in this field are now doing, and Farrell and Saloner [26] give the reason a fragmented ecosystem can persist even where everyone would prefer to converge.

Interoperability profiles are the field’s own answer to the problem we measure, and two of them have worked. The high assurance profile [85] is why ES256 and SD-JWT VC are the two values in Figure 2 that come closest to universal, a success we should credit as readily as we report the gaps. On the other protocol family the Aries Interop Profile [38] did the same job earlier, with a continuously run pairwise test harness [63] of exactly the kind this paper does not have. That both profiles close intersections and neither addresses trust composition is the shape of our argument, not a counterexample to it: Kang et al. [43] formalise why an interoperability suite and a conformance suite do not cover the same ground.

The closest prior empirical work evaluates five wallets against a verifier-side orchestration layer adapting to each ecosystem’s own registry [86]; it resolves fragmentation by translating at the verifier, where we ask what happens when one trust source sits beneath. Nakamura and da Conceição [59] measure integration effort with nine developers across three frameworks, the only existing quantification. Schumm et al. [67] locate the adoption gap in deployment barriers rather than technical constraints.

On software agents, South et al. [69] survey the identity models available to them; Madhira [55] and Tripathi et al. [81] compose authorisation policies by intersection and argue the composition safe; Mellafe Zuvic [56] and Dantuluri and Sundi [12] measure that the common agent frameworks ship without a fail-closed authorisation gate. None composes a registry’s answer about a counterparty with a client’s prior anchor, and none measures what an agent does when that answer is unavailable.

On trust, Krul et al. [48] show it is re-parameterised rather than removed and Konstantinidis et al. [45] name the same phenomenon ecosystem-wide. Cheesman [7] and Giannopoulou [34] reach it from fieldwork and legal theory. De Filippi et al. [13] and Bodó [4] supply the general account of displacement we apply to our own system in §10. Eer et al. [17] propose an alternative registry design and are the closest prior art to Part II. Two queryable registries are already deployed and are not scored in Table 6, because we assessed them from specifications alone

and not against the criteria we applied to the five classes there: the European blockchain infrastructure’s trusted issuer registry [22], which answers accreditation questions within one governance framework, and TRAIN [41], which anchors the pointer to a governing scheme in the naming system rather than in a ledger. Neither settles R2 either, and we did not integrate against them, so the comparison in Part II is architectural rather than measured. Termont and Esteves [78] argue the opposite of our conclusion, that institutional trust lists risk a recentralised and exclusionary ecosystem, and that critique applies to a registry with a governed root as much as to a published list.

The problem is also being worked on inside the standards bodies. Temoshok and Abruzzi [75] set out what a trust framework has to settle beyond protocol, and the W3C is drafting a data model for publishing recognition of issuers and verifiers [42] whose editors have been explicit that they wanted to avoid the vocabulary of lists and authorisation altogether. The research federations reached the same layer earlier and by a different route [51, 64]. What none of these settles is R2: how an answer obtained at runtime composes with whatever the client already believed, which is the question our five composition rules answer five incompatible ways.

14 Conclusion

Thirteen wallets, each making only choices its own specification permits, and four of the five things they would need to agree on have no common value. The fifth has one, and only because four of them cannot do anything else. Keeping only the eleven that share a protocol family, so that the comparison is between systems that were meant to interoperate, leaves three of the five empty. Five incompatible rules govern how they anchor trust, six of them anchor nothing at all, and one binds what it fetches to what it was built with.

Conformance testing asks whether an implementation matches a specification. Interoperability is a property of pairs, and the pairs that are tested are tested for whether a message gets through, not for whether it should have been accepted. The inference from the first to the second does not hold.

The trust question in particular is nobody’s. The profile governing clients says trust is out of scope; the protocol governing registries stops at the wire. In between is where the decision actually gets made, and the wallets we read are each making it up, in five incompatible ways, with a third answer nobody has defined.

A shared queryable trust source addresses part of this. It does not remove the root authority, and our own deployment shows it does not, since we reproduced the thing we set out to replace. What changes is that the list becomes something a stranger can look at, compare, and watch change without waiting for an application update. That is a smaller claim than the one usually made for this kind of infrastructure, and it is the one the evidence supports.

Artifacts

Modified binaries with checksums, exact source diffs for every integration, per-scenario recordings and screenshots, and per-run evidence directories exist for the work in §6 and are being prepared for release; this version of the paper does not yet carry their location, and until it does the integration study should be read as unverified by anyone outside the project. The measurement in §4 does not depend on them. It was taken from public repositories between 2026-07-16 and 2026-08-26, and a reader with those repositories can reconstruct Figure 2 from the resolver registries, key-type tables, format enumerations and manifest entries the method describes. We hold per-cell file and line references and will publish them with the artifacts rather than in the body of the paper.

References

- [1] A2A. Agent2Agent (A2A) protocol specification v1.0: Agent Card signatures. <https://a2a-protocol.org/>, 2026.
- [2] Sumaya Almanee, Arda Ünal, Mathias Payer, and Joshua Garcia. Too quiet in the library: An empirical study of security updates in Android apps’ native code. In *43rd International Conference on Software Engineering (ICSE)*, 2021.
- [3] Christopher J. Atherton, Tom Barton, Jim Basney, Daan Broeder, Antonio Costa, et al. Federated identity management for research collaborations (FIM4R) v2. Technical report, Zenodo, 2018.
- [4] Balázs Bodó. Mediated trust: A theoretical framework to address the trustworthiness of technological trust mediators. *New Media & Society*, 23(9):2668–2690, 2021. doi: 10.1177/1461444820939922.
- [5] Chad Brubaker, Suman Jana, Baishakhi Ray, Sarfraz Khurshid, and Vitaly Shmatikov. Using frankencerts for automated adversarial testing of certificate validation in SSL/TLS implementations. In *2014 IEEE Symposium on Security and Privacy*, 2014. doi: 10.1109/SP.2014.15.
- [6] S. Castro. The relationship between conformance testing of and interoperability between OSI systems. *Computer Standards & Interfaces*, 12(2):97–103, 1991. doi: 10.1016/0920-5489(91)90046-3.
- [7] Margie Cheesman. Self-sovereignty for refugees? the contested horizons of digital identity. *Geopolitics*, 27(1):134–159, 2022. doi: 10.1080/14650045.2020.1823836.
- [8] David Chen, Guy Doumeingts, and François Vernadat. Architectures for enterprise integration and interoperability: Past, present and future. *Computers in Industry*, 59(7):647–659, 2008. doi: 10.1016/j.compind.2007.12.016.
- [9] Yujiao Chen. Trust between AI agents: Measuring formation, breakage, and recovery, with implications for governing multi-agent systems. arXiv:2606.14923, 2026.
- [10] Taejoong Chung, Jay Lok, Balakrishnan Chandrasekaran, David Choffnes, Dave Levin, Bruce M. Maggs, Alan Mislove, John Rula, Nick Sullivan, and Christo Wilson. Is the web ready for OCSP must-staple? In *Proceedings of the 2018 Internet Measurement Conference (IMC)*, 2018. doi: 10.1145/3278532.3278543.
- [11] Jeremy Clark and Paul C. van Oorschot. SoK: SSL and HTTPS: Revisiting past challenges and evaluating certificate trust model enhancements. In *2013 IEEE Symposium on Security and Privacy*, 2013. doi: 10.1109/SP.2013.41.
- [12] Panduranga Sai Varma Dantuluri and Jyotirmoy Sundi. Delegation without trust: An empirical gap analysis of identity, authorization, and runtime governance in multi-agent LLM systems. arXiv:2609.00267, 2026.
- [13] Primavera De Filippi, Morshed Mannan, and Wessel Reijers. Blockchain as a confidence machine: The problem of trust and challenges of governance. *Technology in Society*, 62: 101284, 2020. doi: 10.1016/j.techsoc.2020.101284.
- [14] Joeri de Ruiter and Erik Poll. Protocol state fuzzing of TLS implementations. In *24th USENIX Security Symposium*, pages 193–206, 2015.

- [15] Erik Derr, Sven Bugiel, Sascha Fahl, Yasemin Acar, and Michael Backes. Keep me updated: An empirical study of third-party library updatability on Android. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 2187–2200, 2017. doi: 10.1145/3133956.3134059.
- [16] DIF. did:webvh method specification v1.0. Decentralized Identity Foundation, 2026. <https://identity.foundation/didwebvh/>.
- [17] Kai Jun Eer, Jesus Diaz, and Markulf Kohlweiss. Bottom-up trust registry in self sovereign identity. In *Blockchain and Applications*, 2022. doi: 10.1007/978-3-031-21229-1_39.
- [18] Manuel Egele, David Brumley, Yanick Fratantonio, and Christopher Kruegel. An empirical study of cryptographic misuse in Android applications. In *Proceedings of the 2013 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2013. doi: 10.1145/2508859.2516693.
- [19] ETSI. Electronic signatures and infrastructures (ESI); trusted lists. ETSI TS 119 612 V2.4.1, 2025.
- [20] European Commission. EUDI Wallets Launchpad 2025: key takeaways. European Commission, Brussels, 10–12 December 2025, 2025. <https://ec.europa.eu/digital-building-blocks/sites/spaces/EUDIGITALIDENTITYWALLET/pages/924975727/EUDI+Wallets+Launchpad>.
- [21] European Commission. The european digital identity wallet architecture and reference framework, v3.0.0. European Commission, eIDAS Expert Group, 2026.
- [22] European Commission. EBSI trusted issuers registry API v5. European Blockchain Services Infrastructure, 2026. <https://hub.ebsi.eu/apis/conformance/trusted-issuers-registry>.
- [23] European Commission. Entry/Exit System (EES). European Commission, Directorate-General for Migration and Home Affairs, 2026. https://home-affairs.ec.europa.eu/policies/schengen/smart-borders/entry-exit-system_en, accessed 4 September 2026.
- [24] European Union. Regulation (EU) 2024/1183 amending regulation (EU) no 910/2014 as regards establishing the european digital identity framework. Official Journal of the European Union, 2024.
- [25] Sascha Fahl, Marian Harbach, Thomas Muders, Lars Baumgärtner, Bernd Freisleben, and Matthew Smith. Why Eve and Mallory love Android: An analysis of Android SSL (in)security. In *Proceedings of the 2012 ACM Conference on Computer and Communications Security (CCS)*, pages 50–61, 2012. doi: 10.1145/2382196.2382205.
- [26] Joseph Farrell and Garth Saloner. Standardization, compatibility, and innovation. *The RAND Journal of Economics*, 16(1):70–83, 1985. doi: 10.2307/2555589.
- [27] Joseph Farrell and Garth Saloner. Converters, compatibility, and the control of interfaces. *The Journal of Industrial Economics*, 40(1):9–35, 1992. doi: 10.2307/2950625.
- [28] Daniel Fett, Kristina Yasuda, and Brian Campbell. Selective disclosure for JSON web tokens (SD-JWT). RFC 9901, IETF, 2025.
- [29] Michael P. Gallaher, Alan C. O’Connor, John L. Dettbarn, and Linda T. Gilday. Cost analysis of inadequate interoperability in the U.S. capital facilities industry. Technical Report NIST GCR 04-867, National Institute of Standards and Technology, 2004.

- [30] GDC. Global public-private collaboration makes digital identity standards freely accessible worldwide. Global Digital Collaboration Conference news, 1 September 2026; ISO/IEC 18013 and 23220 series at <https://dinmedia.mdoc.online/>, 2026. <https://globaldigitalcollaboration.org/news>.
- [31] GDC. GDC 2026 Book of Proceedings, session notes (draft): The Pattern That Repeats: Trust Registries Across Every Credential Domain. Collaborative notes, Global Digital Collaboration Conference, Geneva, 2 September 2026, 2026. <https://docs.google.com/document/d/1z365ELPobzYpfnEPEv6XbYe9hAKBUNuA9gIwMmEvn0c>, accessed 4 September 2026.
- [32] GDC. Global Digital Collaboration Conference 2026: programme and session descriptions. Geneva, 1–3 September 2026. <https://globaldigitalcollaboration.org/gdc26?day=sept-1>, <https://globaldigitalcollaboration.org/gdc26?day=sept-2>, <https://globaldigitalcollaboration.org/gdc26?day=sept-3>, 2026. Accessed 4 September 2026.
- [33] Martin Georgiev, Subodh Iyengar, Suman Jana, Rishita Anubhai, Dan Boneh, and Vitaly Shmatikov. The most dangerous code in the world: Validating SSL certificates in non-browser software. In *Proceedings of the 2012 ACM Conference on Computer and Communications Security (CCS)*, pages 38–49, 2012. doi: 10.1145/2382196.2382204.
- [34] Alexandra Giannopoulou. Digital identity infrastructures: A critical approach of self-sovereign identity. *Digital Society*, 2(18), 2023. doi: 10.1007/s44206-023-00049-z.
- [35] Google. Agent Payments Protocol (AP2): specification and agent authorization. <https://github.com/google-agentic-commerce/AP2>, [docs/ap2/specification.md](https://github.com/google-agentic-commerce/docs/ap2/specification.md) and [agent_authorization.md](https://github.com/google-agentic-commerce/docs/ap2/agent_authorization.md); announced 16 September 2025, 2025.
- [36] Michael Grabatin and Wolfgang Hommel. Reliability and scalability improvements to identity federations by managing SAML metadata with distributed ledger technology. In *IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2018. doi: 10.1109/NOMS.2018.8406310.
- [37] GÉANT. eduGAIN technical site: Participating federations. <https://technical.edugain.org/status>, 2026. Accessed 2026-09-04.
- [38] Hyperledger Aries. Aries interop profile (AIP) 2.0. Hyperledger Aries RFC 0302, 2021. <https://github.com/hyperledger/aries-rfcs/tree/main/concepts/0302-aries-interop-profile>.
- [39] ICAO. ICAO Public Key Directory: participants. International Civil Aviation Organization, 2026. <https://www.icao.int/icao-pkd/participants>, 111 participants, accessed 4 September 2026.
- [40] ISO. ISO/IEC 18013-5:2021 personal identification — ISO-compliant driving licence — part 5: Mobile driving licence (mDL) application. International Organization for Standardization, 2021.
- [41] Isaac Henderson Johnson Jeyakumar and Michael Kubach. A trust implementation model for cross-domain decentralized identity ecosystems: architecture, use case, and implementation. *Procedia Computer Science*, 2025. doi: 10.1016/j.procs.2025.02.059.
- [42] Isaac Henderson Johnson Jeyakumar and Manu Sporny. Recognized entities v1.0. W3C Working Draft, 2026. <https://www.w3.org/TR/vc-recognized-entities/>.

- [43] Sungwon Kang, Jinhee Shin, and Myungchul Kim. Interoperability test suite derivation for communication protocols. *Computer Networks*, 35(4):347–364, 2000. doi: 10.1016/S1389-1286(00)00005-0.
- [44] Andor Kesselman, Darrell O’Donnell, Antti Kettunen, and Drummond Reed. ToIP trust registry query protocol (TRQP) specification v2.0. Trust over IP Foundation, Approved Deliverable, 2026. <https://trustoverip.github.io/tswg-trust-registry-protocol/approved/>.
- [45] Ioannis Konstantinidis, Ioannis Mavridis, and Evangelos K. Markakis. The decentralisation paradox in digital identity: Centralising decentralisation with digital wallets?, 2026.
- [46] Nikita Korzhitskii and Niklas Carlsson. Certificate root stores: An area of unity or disparity? In *USENIX Workshop on Cyber Security Experimentation and Test (CSET)*, 2021.
- [47] Mark A. Kramer and Chris Moesel. Interoperability with multiple Fast Healthcare Interoperability Resources (FHIR) profiles and versions. *JAMIA Open*, 6(1), 2023. doi: 10.1093/jamiaopen/ooad001.
- [48] Evan Krul, Hye-young Paik, Sushmita Ruj, and Salil S. Kanhere. SoK: Trusting self-sovereign identity. *Proceedings on Privacy Enhancing Technologies*, 2024(3), 2024. doi: 10.56553/popets-2024-0079.
- [49] Herbert Kubicek, Ralf Cimander, and Hans Jochen Scholl. *Organizational Interoperability in E-Government: Lessons from 77 European Good-Practice Cases*. Springer, 2011. doi: 10.1007/978-3-642-22502-4.
- [50] James Larisch, David Choffnes, Dave Levin, Bruce M. Maggs, Alan Mislove, and Christo Wilson. CRLite: A scalable system for pushing all TLS revocations to all browsers. In *2017 IEEE Symposium on Security and Privacy*, 2017. doi: 10.1109/SP.2017.17.
- [51] Herbert Leitold. Challenges of eID interoperability: The STORK project. In *IFIP Advances in Information and Communication Technology*, volume 352, 2011. doi: 10.1007/978-3-642-20769-3_12.
- [52] Witold Litwin, Leo Mark, and Nick Roussopoulos. Interoperability of multiple autonomous databases. *ACM Computing Surveys*, 22(3):267–293, 1990. doi: 10.1145/96602.96608.
- [53] Yabing Liu, Will Tome, Liang Zhang, David Choffnes, Dave Levin, Bruce Maggs, Alan Mislove, Aaron Schulman, and Christo Wilson. An end-to-end measurement of certificate revocation in the web’s PKI. In *Proceedings of the 2015 Internet Measurement Conference (IMC)*, 2015. doi: 10.1145/2815675.2815685.
- [54] Torsten Lodderstedt, Kristina Yasuda, Tobias Looker, and Paul Bastian. OpenID for verifiable credential issuance 1.0. OpenID Foundation, Final Specification, 2025. https://openid.net/specs/openid-4-verifiable-credential-issuance-1_0.html.
- [55] Partha Madhira. Digital identity for agentic systems: Toward a portable authorization standard for autonomous agents. arXiv:2605.11487, 2026.
- [56] David Mellafe Zuvic. Capability gates are not authorization: Confused-deputy failures in LLM agent frameworks. arXiv:2606.28679, 2026.
- [57] T. Meunier and S. Major. HTTP Message Signatures for automated traffic. Internet-Draft draft-ietf-webbotauth-httpsig-protocol-00, IETF, 1 September 2026, §6.10 and Appendix C.1, 2026.

- [58] Andreas Möller, Stefan Diewald, Luis Roalter, Florian Michahelles, and Matthias Kranz. Update behavior in app markets and security implications: A case study in Google Play. In *Proceedings of the 3rd International Workshop on Research in the Large*, 2012.
- [59] Breno Cerqueira Reis Nakamura and Arlindo Flavio da Conceição. Integration barriers in open-source SSI frameworks: An exploratory developer experience probe, 2026.
- [60] Marten Oltrogge, Yasemin Acar, Sergej Dechand, Matthew Smith, and Sascha Fahl. To pin or not to pin: Helping app developers bullet proof their TLS connections. In *24th USENIX Security Symposium*, pages 239–254, 2015.
- [61] OpenID Foundation. HAIP 1.0 and OpenID4VP 1.0 achieve 98% in OIDF interop testing. OpenID Foundation, 26 November 2025, 2025. <https://openid.net/haip-1-0-openid4vp-1-0-achieve-98-in-oidf-interop-testing/>.
- [62] OpenID Foundation. OpenID federation 1.1. OpenID Foundation, Final Specification, 6 May 2026, 2026. https://openid.net/specs/openid-federation-1_1.html.
- [63] OpenWallet Foundation. Aries agent test harness. OpenWallet Foundation, 2026. <https://github.com/openwallet-foundation/owl-agent-test-harness>.
- [64] Daniela Pöhn, Stefan Metzger, and Wolfgang Hommel. GÉANT-TrustBroker: Dynamic, scalable management of SAML-based inter-federation AAI. In *IFIP International Information Security Conference (SEC)*, 2014. doi: 10.1007/978-3-642-55415-5_25.
- [65] Amogh Pradeep, Muhammad Talha Paracha, Protick Bhowmick, Ali Davanian, Abbas Razaghpanah, Taejoong Chung, Martina Lindorfer, Narseo Vallina-Rodriguez, Dave Levin, and David Choffnes. A comparative analysis of certificate pinning in Android & iOS. In *Proceedings of the 22nd ACM Internet Measurement Conference (IMC)*, pages 605–618, 2022. doi: 10.1145/3517745.3561439.
- [66] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. doi: 10.1109/PROC.1975.9939.
- [67] Daria Schumm, Katharina O. E. Müller, and Burkhard Stiller. Are we there yet? a study of decentralized identity applications. *IEEE Access*, 2025. doi: 10.1109/ACCESS.2025.3588170.
- [68] Juraj Somorovsky, Andreas Mayer, Jörg Schwenk, Marco Kampmann, and Meiko Jensen. On breaking SAML: Be whoever you want to be. In *21st USENIX Security Symposium*, 2012.
- [69] Tobin South, Subramanya Nagabhushanaradhya, Ayesha Dissanayaka, Sarah Cecchetti, George Fletcher, et al. Identity management for agentic AI: The new frontier of authorization, authentication, and security for an AI agent world. OpenID Foundation Artificial Intelligence Identity Management Community Group, arXiv:2510.25819, 2025.
- [70] Manu Sporny, Amy Guy, Markus Sabadello, and Drummond Reed. Decentralized identifiers (DIDs) v1.0. W3C Recommendation, 2022. <https://www.w3.org/TR/did-core/>.
- [71] Manu Sporny, Dave Longley, David Chadwick, and Orie Steele. Verifiable credentials data model v2.0. W3C Recommendation, 2025. <https://www.w3.org/TR/vc-data-model-2.0/>.
- [72] Mike Starling. ‘A systemic failure’: How the new Entry/Exit System (EES) brought chaos to EU border control. Euronews, 14 April 2026, 2026. <https://www.euronews.com/travel/2026/04/14/a-systemic-failure-how-the-new-entryexit-system-ees-brought-chaos-to-eu-border-control>.

- [73] Christoph F. Strnadl. Ecosystem trust profiles, 2026.
- [74] Shigeya Suzuki and Ryosuke Abe. A verifier-centric conceptual model for digital credential ecosystems, 2026.
- [75] David Temoshok and Christine Abruzzi. Developing trust frameworks to support identity federations. Technical Report NISTIR 8149, National Institute of Standards and Technology, 2018.
- [76] Oliver Terbu, Torsten Lodderstedt, Kristina Yasuda, Daniel Fett, and Joseph Heenan. OpenID for verifiable presentations 1.0. OpenID Foundation, Final Specification, 2025. https://openid.net/specs/openid-4-verifiable-presentations-1_0.html.
- [77] Oliver Terbu, Daniel Fett, and Brian Campbell. SD-JWT-based verifiable credentials. Internet-Draft draft-ietf-oauth-sd-jwt-vc-19, in IETF Last Call, 2026. <https://datatracker.ietf.org/doc/draft-ietf-oauth-sd-jwt-vc/>.
- [78] Wouter Termont and Beatriz Esteves. OpenID for european digital identity: An architectural analysis of user-centric identity management, 2026.
- [79] Martin Thomson and David Schinazi. Maintaining robust protocols. RFC 9413, IETF, 2023.
- [80] Andreas Tolk and James A. Muguiru. The levels of conceptual interoperability model. In *Proceedings of the Fall Simulation Interoperability Workshop*. IEEE CS, 2003.
- [81] Bhaskar Tripathi, Anurag Kumar, Ramendra Kumar, and Bhavesh Gadhe. A policy algebra for trust-preserving agentic AI execution. arXiv:2608.16402, 2026.
- [82] Verifiable Intent. Verifiable Intent v0.1: security considerations, section 6.5. Mastercard and Google, 5 March 2026. <https://github.com/agent-intent/verifiable-intent>, 2026.
- [83] Visa. Trusted Agent Protocol: reference implementation. https://github.com/visa/trusted-agent-protocol,merchant-backend/app/security/signature_verification.py and the reference proxy; launched 14 October 2025, 2025.
- [84] Wenguang Wang, Andreas Tolk, and Weiping Wang. The levels of conceptual interoperability model: Applying systems engineering principles to modeling and simulation. In *Proceedings of the Spring Simulation Multiconference (SpringSim '09)*, 2009.
- [85] Kristina Yasuda, Torsten Lodderstedt, Christian Bormann, and Joseph Heenan. OpenID4VC high assurance interoperability profile 1.0. OpenID Foundation, Final Specification, 2025. https://openid.net/specs/openid4vc-high-assurance-interoperability-profile-1_0.html.
- [86] Hakan Yildiz and Axel Küpper. interID: An ecosystem-agnostic verifier application for self-sovereign identity, 2025.
- [87] Hakan Yildiz, Axel Küpper, Dirk Thatmann, Sebastian Göndör, and Patrick Herbke. Toward interoperable self-sovereign identities. *IEEE Access*, 11:114080–114116, 2023. doi: 10.1109/ACCESS.2023.3313723.